

Table of Contents

- Jang
- about
- What this book is not
- FAQs
- Where can I find solutions to the problems in this book ?
- How can I be sure whether a question has a solution or not ?
- Where can I find help ?
- I am unable to solve so-and-so question, why ?
- RESOURCES
- Chapter 1 - Basics
- Chapter 2 - Expressions
- Chapter 3 - Structs
- Chapter 4 - Enum & Patterns
- Chapter 5 - Traits & Generics

The source code of the book is at [link](#)

about

The book is written so that a newbie [Rustaceans](#) can easily dive in the language and get productive as fast as possible

What this book is not

This book is not an attempt to teach you [Rust](#), it simply provides question for practise, and **it is not a tutorial**

FAQs

Where can I find solutions to the problems in this book ?

I have decided not to provide solutions to any problem in this book, you will have to use the long way(irc channels, stackoverflow, etc).

The reason for this is so that you have to study the whole concept rather than a part of it because I feel a lot of new learners simply go and check the solution after being stuck for the first time.

Though help will be provided in some questions as a form of `hint`

How can I be sure whether a question has a solution or not ?

Each and every question that appears in the book has been solved in at least in 1 way before making it's way in the book

Where can I find help ?

Some of the sources that are the most helpful are :

- [Discord Channel for beginners](#)
- [Stackoverflow](#)
- [Reddit community](#)

I am unable to solve so-and-so question, why ?

Mostly because you may be new to the language or to programming world itself, try again and keep on learning, you will be able to eventually solve them with ease

RESOURCES

Some of the sources that might help you in learning [Rust](#)

- [The rust book \(main\)](#) - It is the main book and should be your companion till the end
- [A gentle introduction to rust](#)
- [Learning Rust](#)
- [The official site for Documentation](#)
- [rust-learning](#) - a community maintained learning resources
- [rust-by-example](#) - A practical book to teach you rust

The above resources are more than enough to help you learn your way into the world of Rust

Chapter 1 - Basics

- Write a program to print `Hello, World` .
- Write a program to print a block F using hash (#), where the F has a height of six characters and width of five and four characters.

```
#####  
#  
#  
#####  
#  
#  
#
```

- Create a `tuple` to hold an `i32` and `f32` and then call the `f32` value.
- Take input(name) from the user of type `String` and print `Hello, <name>!` .
- Initialize an array of sixteen `0`

Chapter 2 - Expressions

- Run an infinite `loop` to keep on printing `Hello, World!`.
`hint:` you might want to use `ctrl+c` or `ctrl+z` to exit the infinite loop.
- Take a `integer` input from the user and print table for that integer using a loop.
- Print the following pattern using loops.

```

    *
   ***
  *****
 *****
*****

```

- Check whether the input number is odd or even and print `odd` or `even` respectively.
- Find and print factorial of a program using recursion.
- Using a match statement to print `one` for input `1`, `two` for `2` and so on, and `NaN` on default.
- Create a diverging function for addition of 2 numbers.
- Create a program to check for leap year.
`note:` 1900 is not a leap year.
- Write a function to swap 2 numbers.
- Write a program to find prime numbers upto a number `N`
- WAP to sort a list of numbers.
- WAP to iterate over 2 vectors at once.
`hint:` try using `.zip` method.

Chapter 3 - Structs

- Write a program to store and print the roll no., name , age and marks of a student using structures.
- Write a program to compare two dates entered by user. Make a structure named Date to store the elements day, month and year to store the dates. If the dates are equal, display "Dates are equal" otherwise display "Dates are not equal".
- Write a program to add, subtract and multiply two complex numbers using structures to function.
- Create a structure named Date having day, month and year as its elements. Store the current date in the structure. Now add 45 days to the current date and display the final date.
- Replicate constructor function with name `new()` returning type `Self`.
- Let us work on the menu of a library. Create a structure containing book information like accession number, name of author, book title and flag to know whether book is issued or not. Create a menu in which the following can be done.
 - 1 - Display book information
 - 2 - Add a new book
 - 3 - Display all the books in the library of a particular author
 - 4 - Display the number of books of a particular title
 - 5 - Display the total number of books in the library
 - 6 - Issue a book

(If we issue a book, then its number gets decreased by 1 and if we add a book, its number gets increased by 1)
- Create a generic struct for addition of numbers (they can be `integer` or `float`).
- Create a struct with `Copy, Clone, Debug, PartialEq` traits
hint: Use `#[derive]`
- Create an immutable struct with a mutable member.
hint: Use `Cell`, property known as interior mutability.
- Try making the above program with `RefCell` so that the struct stores details about a bank(balance, customer count, location, etc) with only customer count being mutable.

Chapter 4 - Enum & Patterns

- Create an enum that contains HTTP 4xx Client Errors.
i.e. `Not Found` Should have value `404` associated with it.
- Write an enum to store information of whether a person is a child or adult based on his/her age.
- Create a calculator with the help of an enum named `Operation` that as values such as `Add`, `Sub`, `Div`, `Multi`.
hint: For input like `2+5*10` it should first evaluate `5*10` and then add 2 to it.
- Use pattern matching to associate a enum of `Grade` type with a student based on his/her marks.
- Create an enum named `Json` that can work with arbitrary JSON data.
- Print what kind of input the user has given(numbers, letters, symbols) using pattern matching.
hint: Try using range for it e.g. `0 ... 100` or `'a' ... 'k'`
- Use pattern matching to find that whether a point lies on X-Axis, Y-Axis or on which quadrant.
- Create a struct that holds info about a gun(gun type, recoil time, magazine size, extra) where `gun type` and `extra` are enums and `extra` contains silencer, scope, extended mags nad `None`. Based on user input change the value of extra (may cause change in recoil time and magazine size).
- Create a Binary tree and have a method `add` on it to add elements to it and `min` and `max` to find the minimum and maximum element in it.
- Create a `Regex` to extract dates from this string `It was on 2019-03-14, almost after a year from 2018-02-11` and store in a Struct with fields of `day`, `month` and `year`.
hint: Use `Regex` crate

Chapter 5 - Traits & Generics

- Derive a debug trait to print info about your struct that contains `name`, `class` and `roll`.
- Create a generic function to get min of 2 values.
`hint:` You might need to use `Ord` trait bound.
- Implement custom `Drop` trait for a struct name `Student` that contains your `name`, `age` and `roll number`. It should return `Roll number <roll number> has name <name> with age <age> and is a <junior/senior>`. Being Junior or Senior depends on age (18 or above).
- Implement a custom `Debug` message for the above Struct.
- Implement custom `Iterator` trait for a struct named `GeometricSeries` that has 3 fields `first_number`, `current_number` and `ratio` of type `i32`. The iterator should return the next 11 numbers in geometric progression.
`hint:` Use `.take(11)` to get the next 11 in `for` loop.
- Implement the same as above for a `FibonacciSeries` struct.
- Implement a generic function name `sum` with additional parameter of `index: usize` that can take either `GeometricSeries` or `FibonacciSeries` and returns the sum upto the given index.
`hint:` use `T: Iterator<Item = i32>` where `T` is generic
- Write a generic function name `Multiply` that multiplies all `usize`, `isize` and `fsize` type values.
- Make a generic function to return the greater of the 2 given values (integer and floats).
- Implement a trait named `Hello` with a default function `say(&self)` that prints `Hello, <self>!`, Implement this for `str` and `string` without providing any definition of `Hello` (simply `impl Hello for str {}`) call `say` on `str World`.

- Create a struct name `Set` that contains a `Vector` of chars and overload the `minus` operator so that when 2 structs subtract it removes the chars of 2nd from 1st one.
- Create a struct named `ComplexNumber` that has 2 variables `re` & `im` and overload `minus` and `plus` operator to add and subtract complex number.
- Overload the `!` operator to conjugate the complex number `!ComplexNumber` and `==` and `!=` for comparison.
- Create a struct named `Class` that contains the class size, section and grade. Overload the `>`, `<`, `>=`, `<=`, `==` operators to compare class sizes of various Classes.
- Rust does not allow addition of `integer` and `float`. Overload `+` so that this is possible.
- Implement custom `Drop` for a struct `Hero` that contains a field `name` of type string. The `drop` should print "Oh no !!! Our hero {name} is defeated". Run the program with just declaring a variable of type `Hero`.
- Create the struct named `World` that contains the previous named struct `Hero`, define `drop` for it so that it prints "The world ends here !!!". Observe the order in which `World` and `Hero` contained by it are dropped.
- Create a struct named `Table` that has a generic field `legs_info` of type `T`. Create a function (not a method) named `show` that accepts function parameters `&Table<Display>` and displays `legs_info`. Create 2 variables one that contains `T` of type `String: "Work in progress..."` and `usize: 4`.
`hint:` use `?Sized`.
- Implement `From` trait for struct `ComplexNumber` given a tuple of type `(isize, isize)`. Form it using `from` and `into` respectively.
- Create a function that accepts either `ComplexNumber` or `(isize, isize)` to return the `mod` of `ComplexNumber`.

- Create a closure `add_one` to add 1 to an integer.
- Create an vec of numbers from `0` ... `100` , mutate the vec so that it does not contain any number divisible by `3` . Use `retain` method of Vec.
- Create a function that accepts a closure

- Implement a macro named `addition` to add any amount of numbers. Eg: `addition!(5, 6, 57, 56, 1)` should return `125` and `addition!(4, 9)` should return `11`.

